Sắp xếp và độ phức tạp

Tài liệu học tập chi tiết · Lý thuyết, ví dụ, thực hành và hướng giải

Vị trí bài học trong khóa

- Khóa học: Cấu trúc dữ liệu và giải thuật cơ bản.
- Chương 2: Tìm kiếm và sắp xếp.
- Các bài trong chương: Tìm kiếm tuyến tính và nhị phân; Sắp xếp và độ phức tạp.
- Đọc lý thuyết, tự làm ví dụ rồi đối chiếu hướng giải; sau đó mới làm quiz. Bài thực hành tổng hợp cuối khóa dùng để kết nối kiến thức giữa các chương.

> ### Mục tiêu bài học
>
> Đọc Big-O và mô tả hai thuật toán sắp xếp cơ bản.

Kiến thức nền

Big-O mô tả một chặn trên tiệm cận của chi phí theo kích thước đầu vào. O(n) tăng tuyến tính, $O(n^2)$ tăng theo bình phương. Khi so sánh cần cùng loại chi phí và trường hợp phân tích; không suy ra thời gian thực tế chỉ từ Big-O.

Selection sort chọn phần tử nhỏ nhất trong phần chưa sắp xếp rồi đưa về đầu phần đó. Thuật toán thường có $O(n^2)$ phép so sánh và có thể thực hiện tại chỗ. Merge sort chia dữ liệu, sắp xếp các nửa rồi trộn hai dãy đã có thứ tự; thời gian thường O(n log n).

Bản merge sort thông thường trên mảng cần O(n) bộ nhớ phụ. Một thuật toán ổn định giữ thứ tự tương đối của các phần tử có khóa bằng nhau. Ổn định và tại chỗ là hai thuộc tính khác nhau, không phải tên gọi của độ phức tạp thời gian.

1. Đặt tiêu chí trước khi chọn thuật toán

Đầu ra tăng hay giảm, có giữ phần tử bằng nhau theo thứ tự cũ không, có sửa đầu vào không, giới hạn bộ nhớ bao nhiêu? Một thuật toán tại chỗ và một thuật toán ổn định có thể phù hợp các nhu cầu khác nhau. So sánh chi phí phải cùng mô hình dữ liệu.

2. Merge sort đầy đủ cho danh sách số

```
def merge_sort(values):
    if len(values) <= 1:
        return values[:]
    middle = len(values) // 2
    left = merge_sort(values[:middle])
    right = merge_sort(values[middle:])
```

```
    output = []
    i = j = 0
    while i < len(left) and j < len(right):
        if left[i] <= right[j]:
            output.append(left[i]); i += 1
        else:
            output.append(right[j]); j += 1
    return output + left[i:] + right[j:]

assert merge_sort([4, 2, 5, 1]) == [1, 2, 4, 5]
assert merge_sort([2, 2, -1]) == [-1, 2, 2]
```

Chọn bên trái khi khóa bằng giúp giữ ổn định nếu phép so sánh theo khóa nhất quán. Bản ví dụ dùng slicing và danh sách phụ để dễ hiểu, không phải bản tối ưu bộ nhớ.

3. Vì sao O(n log n)

Chia đôi tạo khoảng log n mức; công việc trộn trên mỗi mức là O(n). Không ghi riêng một lần trộn rồi kết luận toàn thuật toán O(n). Với bản cài thông thường, bộ nhớ phụ cực đại là O(n) cộng ngăn xếp O(log n), vẫn O(n).

4. Bài tập và lời giải

Trộn [1,4,7] với [2,3,8] thu [1,2,3,4,7,8]; ở mỗi bước chọn đầu nhỏ hơn chưa dùng. Kiểm tra đầu ra vừa có thứ tự vừa giữ số lần xuất hiện từng phần tử. Một chương trình trả [] có thứ tự nhưng đã mất dữ liệu, nên chỉ kiểm tra tăng dần là chưa đủ.

> **Bài tập vận dụng bổ sung**
>
> mô phỏng hai bước selection sort cho [4,2,5,1]. Trộn [1,4,7] với [2,3,8]. Với bản ghi cùng điểm, giải thích vì sao giữ thứ tự ban đầu có thể hữu ích.

Câu hỏi tự kiểm tra

1. $O(n^2)$ thường mô tả mức tăng chi phí như thế nào?
2. Selection sort thường chọn gì ở mỗi bước?
3. Thời gian thông thường của merge sort là gì?
4. Sắp xếp ổn định có ý nghĩa gì?
5. Merge sort thông thường trên mảng thường cần bộ nhớ phụ bao nhiêu?

Gợi ý đáp án và điểm cần nhớ

- Theo bình phương kích thước đầu vào. Big-O biểu diễn quan hệ tăng trưởng, không phải thời gian tuyệt đối.
- Phần tử nhỏ nhất trong phần chưa sắp xếp. Phần tử nhỏ nhất được đưa về đầu phần chưa sắp xếp.

- O(n log n). Có khoảng log n mức chia và O(n) công việc trộn mỗi mức.
- Giữ thứ tự tương đối của phần tử có khóa bằng nhau. Tính ổn định xét các phần tử có khóa so sánh bằng nhau.
- O(n). Bộ đệm trộn thông thường có kích thước tỉ lệ với số phần tử.

Cách tự đánh giá sau bài

- Giải thích được thuật ngữ mà không đọc lại định nghĩa.
- Thực hiện lại ví dụ với dữ liệu khác và nêu được giới hạn áp dụng.
- Hoàn thành bài tập, đối chiếu bằng chứng hoặc đầu ra với hướng giải.
- Nếu còn nhầm, ghi câu hỏi cụ thể để hỏi giảng viên hoặc AI Tutor cùng ngữ cảnh bài.

Các câu tự kiểm tra có thể trùng trọng tâm quiz. Làm đúng câu đã đọc không tự chứng nhận thành thạo; cần thêm bài vận dụng và minh chứng thực hành.

Tài liệu đọc thêm

- https://opendatastructures.org/
- https://docs.python.org/3/tutorial/datastructures.html
- https://docs.python.org/3/library/heapq.html

Sắp xếp và độ phức tạp — Thực hành và hỏi đáp

Phần bổ sung cho bài học · biên soạn 29/10/2026

Tình huống, ví dụ và tự kiểm tra

Tình huống: muốn sắp xếp sản phẩm theo giá rồi tên. Xác định khóa và việc có được sửa dữ liệu đầu vào hay không.

```
items = [(2, "B"), (1, "C"), (1, "A")]
ordered = sorted(items, key=lambda item: (item[0], item[1]))
assert ordered == [(1, "A"), (1, "C"), (2, "B")]
assert items[0] == (2, "B")
```

Tự luyện: dữ liệu đã tăng, giảm, trùng và rỗng. **Hướng giải:** kiểm tra bảo toàn số phần tử và tính có thứ tự. Với thuật toán tự viết, nêu worst/average/best khi cần; không kết luận O(n) chỉ từ một input đã sắp xếp.

Nguồn học tiếp

https://docs.python.org/3/library/collections.html